Intro to R - Ggplot2

B.A. Hartlaub, A.S. Wagaman, and I. Sanchez

Introduction

When working in R, there are several available graphics packages. Base R graphics are the built in ones such as *hist* and *plot*. The graphics are functional but not amazing. You may have seen *lattice* graphics, which was an upgrade over base R graphics. The most recent graphics package that is heavily used is *ggplot2*. In our examples, we've been using a wrapper to *ggplot2* called *ggformula* for many plots because it makes the syntax easier. At some point though, you may want to explore *ggplot2*, and that's what this document is designed for.

This file follows the Intro to R - Common Commands document, but shows how to generate those graphs with *ggplot2* syntax, rather than *ggformula*.

In ggplot2, the basic syntax to build a graph is as follows:

```
# ggplot(DATASET,
       # aes(x = XVAR,
           # y = YVAR,
           # ANY OTHER AESTHETIC PROPERTIES TO BE ADDED TO ALL LAYERS)) +
  # geom_PLOTTYPE(aes(ANY OTHER AESTHETIC PROPERTIES TO BE ADDED TO THIS LAYER ONLY)) +
  # other options - LAYER, COORDINATE, POSITION, FACETING, ETC.
  # be sure to add a '+' between each layer/function type
```

Note that you need several components - a data set, your variables (though it can be just one), and your aesthetics (aes), such as shape, color, etc. in an initial call to ggplot. The graph then adds *layers* to the plot that plot the various pieces, such as geom_point(), which adds points to the plot. The commands use the plus sign for different lines.

For more plots and customization features, see the ggplot2 cheatsheet.

Packages

Let's load a few libraries to help us through these examples.

```
library(ggplot2)    # Grammar of Graphics plotting system
library(mosaic)     # Simplified syntax for modeling, stats, and graphics
library(NSM3)       # Nonparametric statistical methods
library(tidyverse) # Collection of data science packages
library(Stat2Data) # Datasets
```

Remember we are following the Intro to R framework below, but just doing the graphs differently. If you want a refresher on the data sets, etc., you should look that up now.

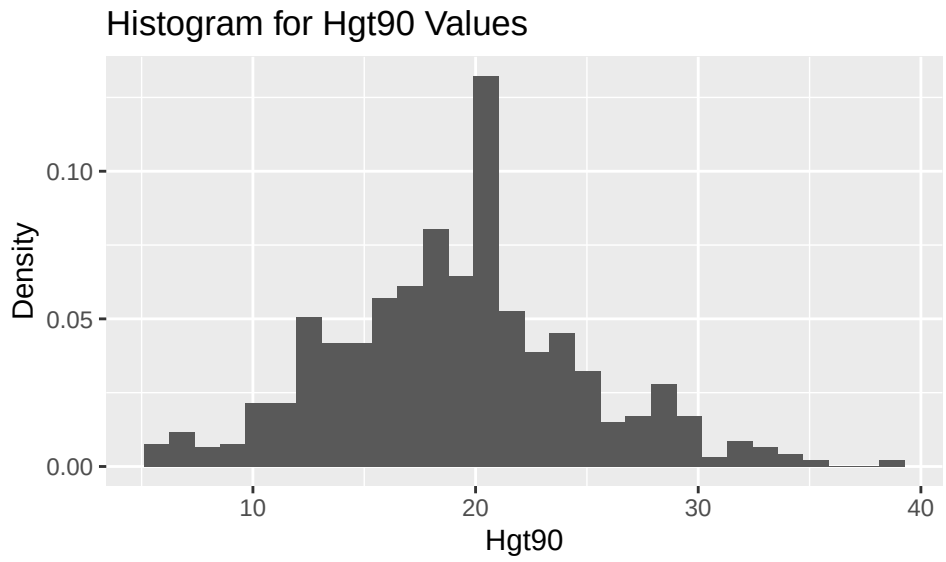
Univariate Exploration - Quantitative

```
data(Pines)
```

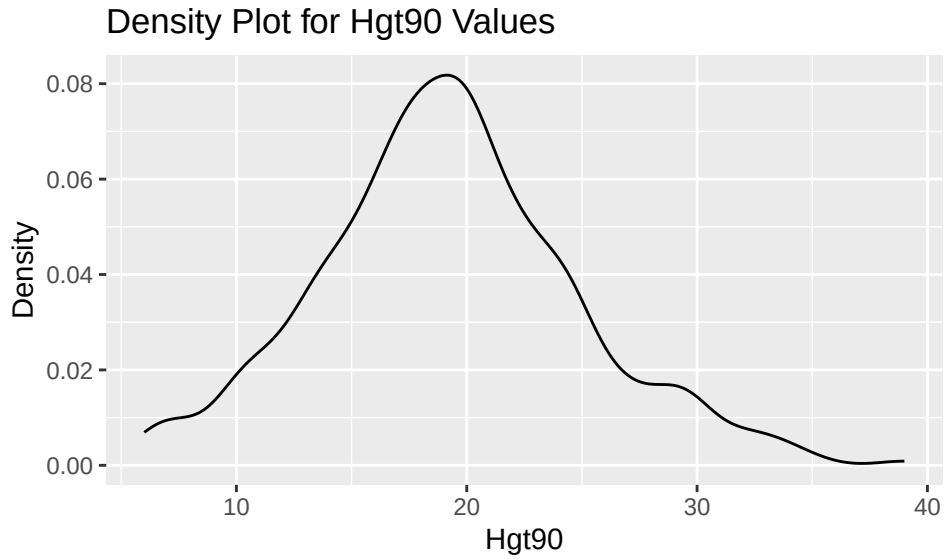
For a graphical description of the distribution of a single variable, we can create a histogram or a density plot.

```
# default histogram for Hgt90
ggplot(Pines, aes(x = Hgt90)) +
  geom_histogram(aes(y = after_stat(density))) +
  labs(title = "Histogram for Hgt90 Values",
       x = "Hgt90", y = "Density")
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

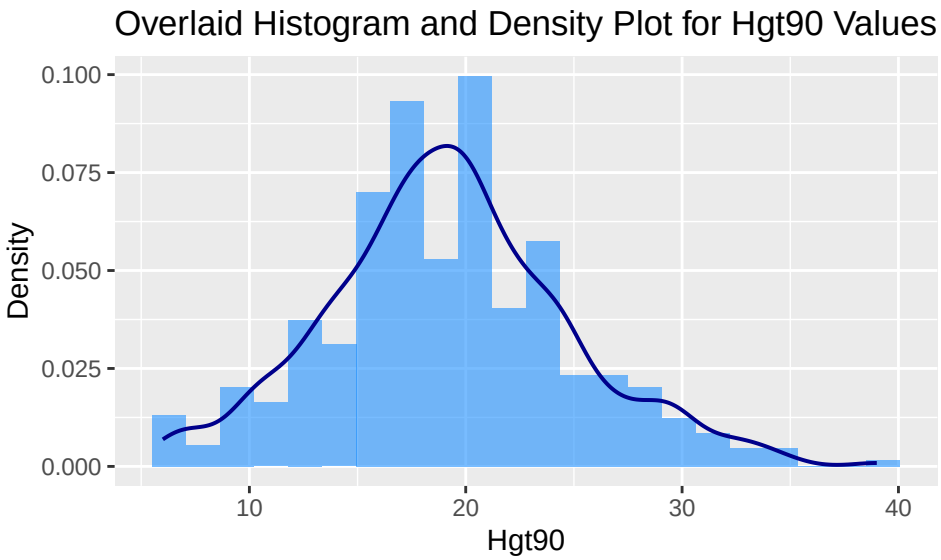



```
# default density plot for Hgt90
ggplot(Pines, aes(x = Hgt90)) +
  geom_density() +
  labs(title = "Density Plot for Hgt90 Values",
       x = "Hgt90", y = "Density")
```




```
# overlaid histogram
# showing some of the customization features including:
```

```
# color, transparency, bins, and line linewidth
ggplot(Pines, aes(x = Hgt90)) +
  geom_histogram(aes(y = after_stat(density)),
                 fill = "dodgerblue",
                 alpha = 0.6,
                 bins = 22) +
  geom_density(color = "blue4", linewidth = 0.7) +
  labs(title = "Overlaid Histogram and Density Plot for Hgt90 Values",
       x = "Hgt90", y = "Density")
```



Remember, if you want to use this code, go ahead! Copy and paste the code you have as an example and edit the variables and data set to your situation.

Univariate Exploration - Categorical

If the variable you are interested in is a categorical variable, you might try a table of counts via *tally* or a bar plot as part of your univariate analysis.

Our **Pines** data set has some categorical variables but all are coded numerically. Briefly, we show how to set one as a categorical variable to demonstrate the univariate analysis.

The *Fert* and *Spacing* variables are binary variables representing whether fertilizer was used and the spacing between trees. We use the mutate command to make them factors (so R understands they are categorical).

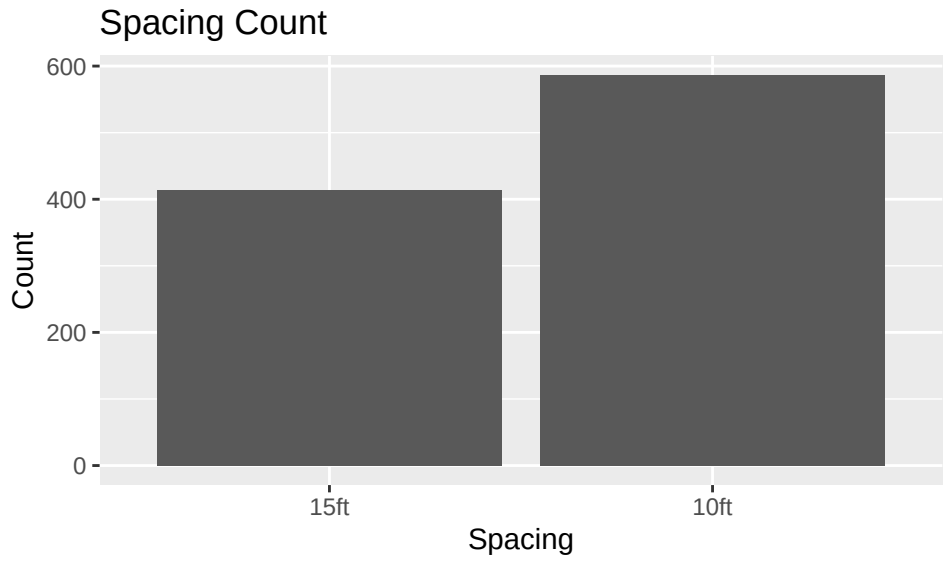
```
Pines <- mutate(Pines,
                Fert = factor(Fert, levels = c(0, 1),
                              labels = c("No", "Yes")),
                Spacing = factor(Spacing, levels = c(15, 10),
                                 labels = c("15ft", "10ft")))
```

Now, let's do a univariate examination of the *Spacing* variable.

```
tally(~ Spacing, data = Pines)
```

```
## Spacing
## 15ft 10ft
##  414  586
```

```r
ggplot(Pines, aes(x = Spacing)) +
  geom_bar() +
  labs(title = "Spacing Count",
       y = "Count")
```

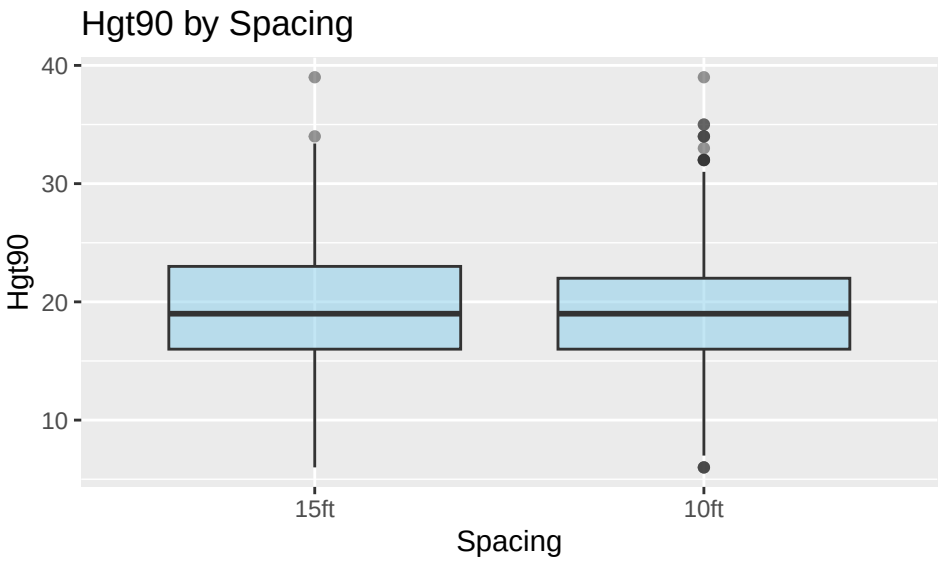


Bivariate Analysis

Often, we examine more than one variable at a time to investigate their relationships. In that case, a variety of plots may be useful, depending on the types of variables involved.

For relating a quantitative variable to a categorical variable, we can draw side-by-side boxplots. This plot, for example, allows us to look at the height of the pine trees at planting and if there were differences based on spacing. (You should note that since is the start of an experiment, we don't want to see differences here.)

```r
ggplot(Pines, aes(x = Spacing, y = Hgt90)) +
  geom_boxplot(alpha = 0.5, fill = "skyblue") +
  labs(title = "Hgt90 by Spacing")
```



Descriptive statistics by group can also be useful here. Note the quantitative variable goes before the tilde, and the categorical variable goes after.

```
favstats(Hgt90 ~ Spacing, data = Pines)
```

```
##   Spacing min Q1 median Q3 max     mean       sd   n missing
## 1    15ft   6 16     19 23  39 19.31281 5.731915 320      94
## 2    10ft   6 16     19 22  39 19.12851 5.575232 498      88
```

What if both variables involved are categorical? We can make a contingency table using tally to look at the values, or scale to proportions or percents as desired. There are also plot options you could explore using other R packages (not shown).

```
tally(~ Spacing + Fert, data = Pines, margins = TRUE) # raw counts
```

```
##        Fert
## Spacing   No  Yes Total
##   15ft   208  206   414
##   10ft   285  301   586
##   Total  493  507  1000
```

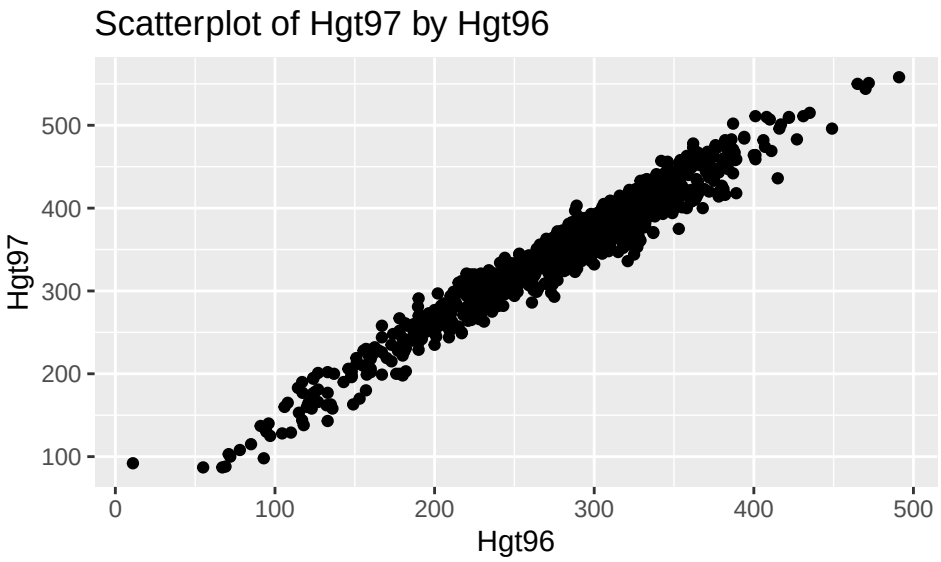
```
tally(~ Spacing + Fert, data = Pines, margins = TRUE, format = "percent")
```

```
##        Fert
## Spacing    No   Yes Total
##   15ft   20.8  20.6  41.4
##   10ft   28.5  30.1  58.6
##   Total  49.3  50.7 100.0
```

Again, considering the context, *Fert* and *Spacing* should be independent. Does the table support this?

Finally, if the two variables involved are quantitative, we typically make a scatterplot. If appropriate, you can compute correlation and fit a linear regression line. Here, we might naturally assume there is a relationship between *Hgt96* and *Hgt97*, so let's examine those a bit more closely.

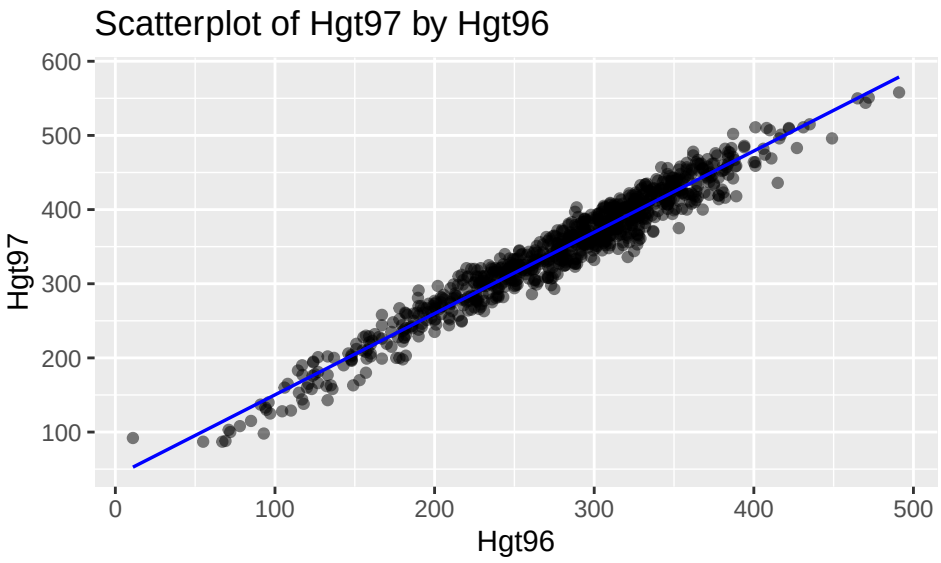
```
ggplot(Pines, aes(x = Hgt96, y = Hgt97)) +
  geom_point() +
  labs(title = "Scatterplot of Hgt97 by Hgt96")
```



This shows a strong positive linear relationship, so we can add the regression line to this plot.

```
ggplot(Pines, aes(x = Hgt96, y = Hgt97)) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm", color = "blue", linewidth = 0.6, se = FALSE) +
  # adds the regression line
  # 'se = FALSE' removes the shaded SE region
  labs(title = "Scatterplot of Hgt97 by Hgt96")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

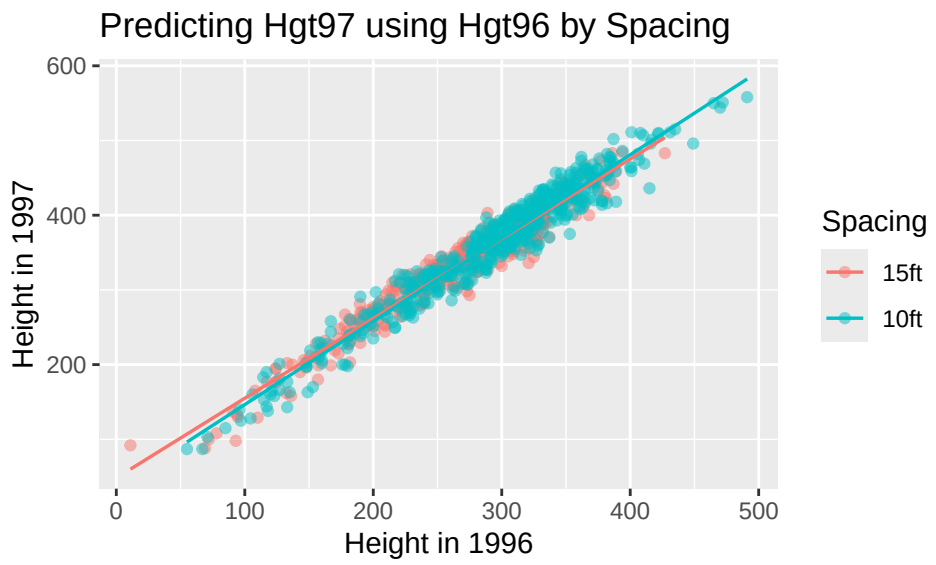


Note that there are many graphical options that we can add to this plot - some of which add more variables. Here is the same basic plot, but jazzed up a bit. Feel free to experiment with graphical options! If you don't know what is available, hit the tab key inside the function call. Or use the Help menu options. Remember to use examples to help you get code that works for what you want.

```
# adds categorical coloring
ggplot(Pines, aes(x = Hgt96, y = Hgt97, color = Spacing)) +
```

```
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm", , linewidth = 0.6, se = FALSE) +
  labs(title = "Predicting Hgt97 using Hgt96 by Spacing",
       x = "Height in 1996",
       y = "Height in 1997")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

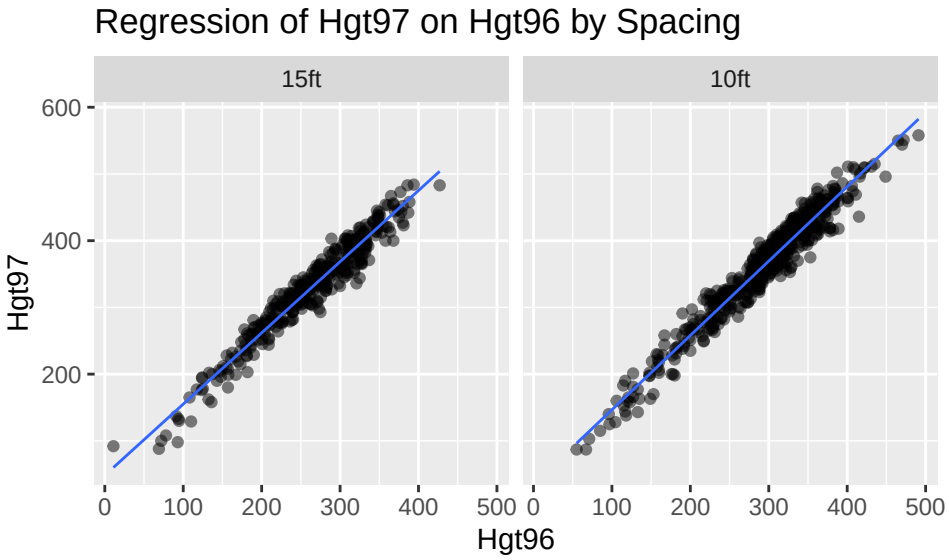


Note that in this case we have conditioned on a third variable *Spacing*, via the color argument.

We could also display the third variable *Spacing* through side-by-side plots.

```
ggplot(Pines, aes(x = Hgt96, y = Hgt97)) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm", linewidth = 0.5, se = FALSE) +
  facet_wrap(~ Spacing) + # organizes plots into panels
  labs(title = "Regression of Hgt97 on Hgt96 by Spacing")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Conditions

Based on your work so far, a simple linear regression model is reasonable, so let's look at fitting the model and getting plots to check our conditions as our last bit of code.

Note that *fm* stands for fitted model, but you can use any name you want for the model. We use the assignment operator to "save" the model because we call it in several later commands. Common names for models you might see in class are things like *mod*, *mod1*, *mymod*, etc. Just be sure to keep track of what name you are using for calling the summaries and other related commands.

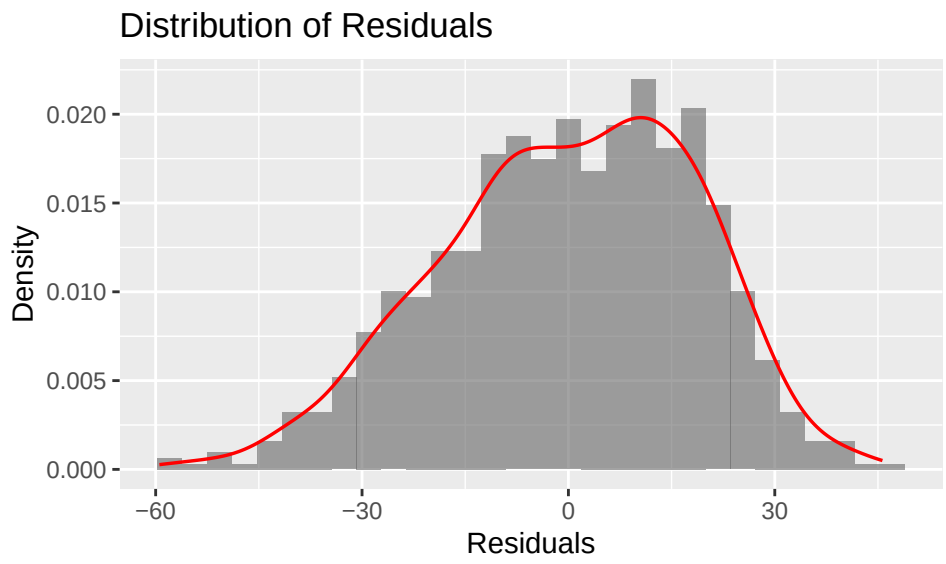
```
fm <- lm(Hgt97 ~ Hgt96, data = Pines)
```

How do we check conditions? There are several ways to get the necessary plots. Some are easier than others. To plot these using ggplot2, we have to save the components we need, though if you are using mosaic, the built-in plots should be rendered in ggplot2 already.

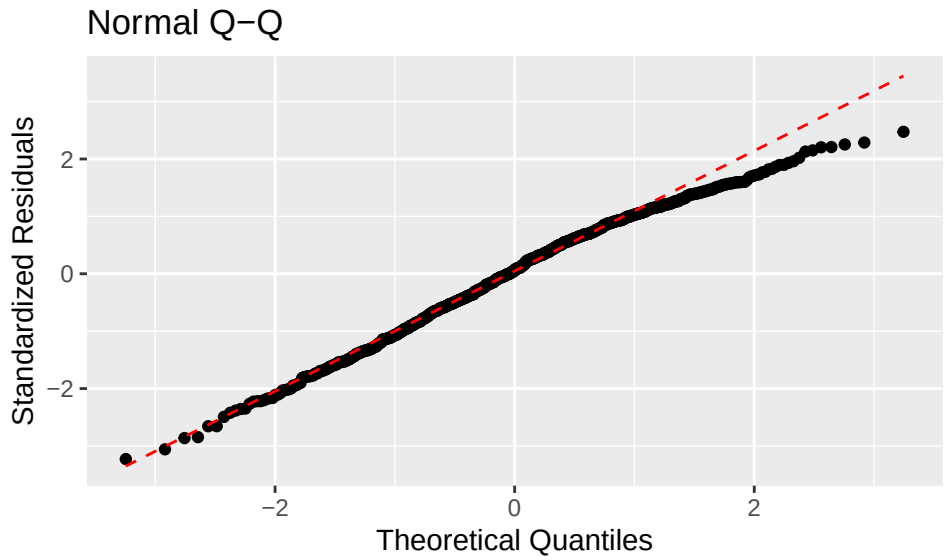
```
# calculate and store the residuals
residuals_df <- data.frame(residuals = residuals(fm),
                           std_res = rstandard(fm),
                           fitted = fitted(fm)) # adds standardized residuals
```

```
# generates histogram with density curve
ggplot(residuals_df, aes(x = residuals)) +
  geom_histogram(aes(y = after_stat(density)), fill = "gray40", alpha = 0.6) +
  geom_density(color = "red", linewidth = 0.6) +
  labs(x = "Residuals",
       y = "Density",
       title = "Distribution of Residuals")
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



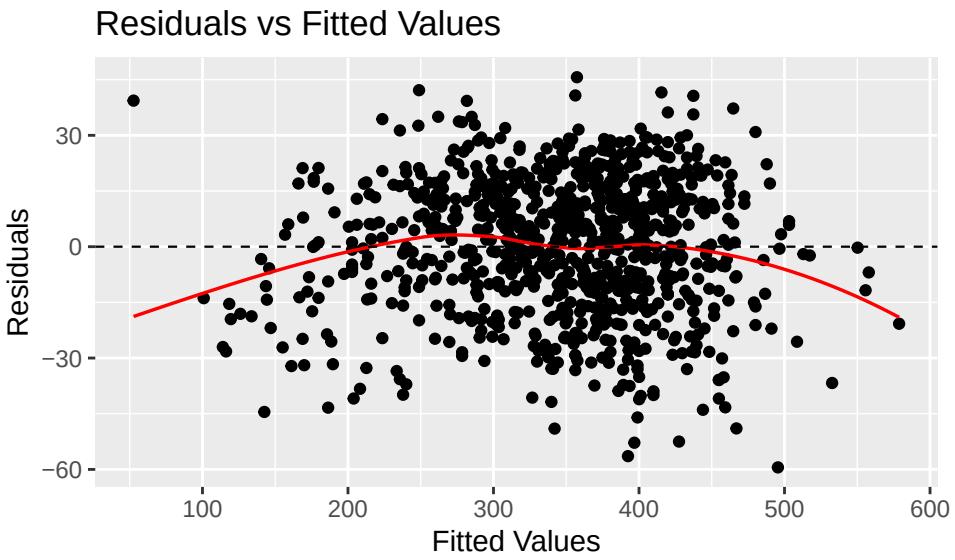

```
# generates the associated qqplot
ggplot(residuals_df, aes(sample = std_res)) +
  stat_qq() +
  stat_qq_line(color = "red", linetype = "dashed") +
  labs(title = "Normal Q-Q",
       x = "Theoretical Quantiles", y = "Standardized Residuals")
```



Constant Variance condition:

```
ggplot(residuals_df, aes(x = fitted, y = residuals)) +
  geom_point() +
  geom_smooth(color = "red", se = FALSE, linewidth = 0.6) +
  geom_hline(yintercept = 0, linetype = "dashed", linewidth = 0.4) +
  # creates the middle dashed line
  labs(title = "Residuals vs Fitted Values",
       x = "Fitted Values", y = "Residuals")
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



We hope these examples are useful if you decide to explore the use of ggplot2 for graphics!

Standard Reference

These materials were created for use in an undergraduate nonparametrics course. Where provided, textbook references refer to Nonparametric Statistical Methods (3rd Edition) by Hollander, Wolfe, and Chicken. Notation is chosen to be consistent with that text. However, materials may be used independently of the textbook.

License